

Physics Research Note1

Jizheng Bo

April 23, 2026

Contents

Chapter 1 Proton Decay 的 Bayesian 计算	5
1.1 试运行	5
1.2 正式运行	15
Appendix A Supplementary Calculations	27

Chapter 1

Proton Decay 的 Bayesian 计算

考虑形变势能:

$$V(r, \theta) = V_N(r, \theta) + V_c(r, \theta) + V_{so}(r, \theta) + \frac{\hbar^2}{2\mu r^2} \left(l + \frac{1}{2}\right)^2 \quad (1.1)$$

$$V_N(r, \theta) = -\frac{V_{N0}}{1 + \exp\{r - R_N(\theta)/a_N(\theta)\}} \quad (1.2)$$

$$R_N(\theta) = R_{N0}(1 + \beta_2 Y_{20}(\theta) + \beta_4 Y_{40}(\theta)) \quad (1.3)$$

$$a_N(\theta) = a_{N0} \sqrt{1 + \left(\frac{1}{R_N(\theta)} \frac{dR_N(\theta)}{d\theta}\right)^2} \times [1 + \tilde{\beta}_2 Y_{20}(\theta) + \tilde{\beta}_4 Y_{40}(\theta)] \quad (1.4)$$

$$V_{so} = V_{so0} \lambda_\pi^2 \frac{1}{r} \frac{d}{dr} \frac{1}{1 + \exp[(r - R_{so})/a_{so}]} \vec{\sigma} \cdot \vec{l} \quad (1.5)$$

$$\lambda_\pi^2 = 2fm^2 \quad (1.6)$$

$$V_c(r) = \begin{cases} \frac{Z_D Z_p e^2}{2R_{c0}} \frac{3R_{c0}^2 - r^2}{R_{c0}^2}, & r \leq R_{c0} \\ \frac{Z_D Z_p e^2}{r}, & r > R_{c0} \end{cases} \quad (1.7)$$

1.1 试运行

使用参数 (145Tm, 形变势能, wkb 积分)

```
#System
e2=1.43997 ; hbarc=197.3269718 ; amu=931.49432
zp=69 ; Ap=145 ; mass_excess_p=-27.58 ; mp=Ap*amu+mass_excess_p
z1=68 ; A1=144 ; mass_excess_1=-36.61 ; m1=A1*amu+mass_excess_1 #1-> daughter
z2=1 ; A2=1 ; mass_excess_2=7.288971064 ; m2=A2*amu+mass_excess_2
Q=mp-m1-m2
mu=m1*m2/(m1+m2)
P0=0.580
#Potential
```

```

#Deformation
beta2 = 0.231 # quadrupole
beta4 = -0.068 # hexadecapole

#Polarizaton
beta2tilde = 0
beta4tilde = 0

#spin-orbit
Vso0=6.2 ; rso=1.01*(A1)**(1/3) ; aso=0.75 ; lambda_pi=np.sqrt(2)
L=5; S=0.5; J=5.5

#Vn
a0=0.75 ; r0=1.27*A1**(1/3)-0.1 ; V0=55

#Vc
Rc0 = 1.21 * A1**(1/3)

# experiment data
y=np.array([3.16e-6]) # s
sigma=np.array([0.16e-6])

#a = v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso
def log_prior(a):
    v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso=a
    R=1

# Hyperparameters
v0_mean=55; a0_mean=0.75; r0_mean=6.556; p0_mean=0.580
rc0_mean=6.342; beta2_mean=0.231; beta4_mean=-0.068
vso0_mean=6.2; rso_mean=5.294; aso_mean=0.75

sigma_v0=1*R; sigma_a0=0.05*R; sigma_r0=0.1*R; sigma_p0=0.05*R
sigma_rc0=0.1*R; sigma_beta2=0.05*R; sigma_beta4=0.01*R
sigma_vso0=1*R; sigma_rso=0.1*R; sigma_aso=0.05*R

with multiprocessing.Pool() as pool:
    sampler = emcee.EnsembleSampler(nwalkers, M, log_posterior, args=[y,sigma],a=1.0,
    pool=pool)
    state = sampler.run_mcmc(initial_positions, 200)
    sampler.reset()

```

```
sampler.run_mcmc(state,10000)
```

获得 ^{145}Tm 的 Posterior 分布如下图所示:

(说明之前的 bug 出自于对 model 函数的解包错误, 而非采样参数错误)

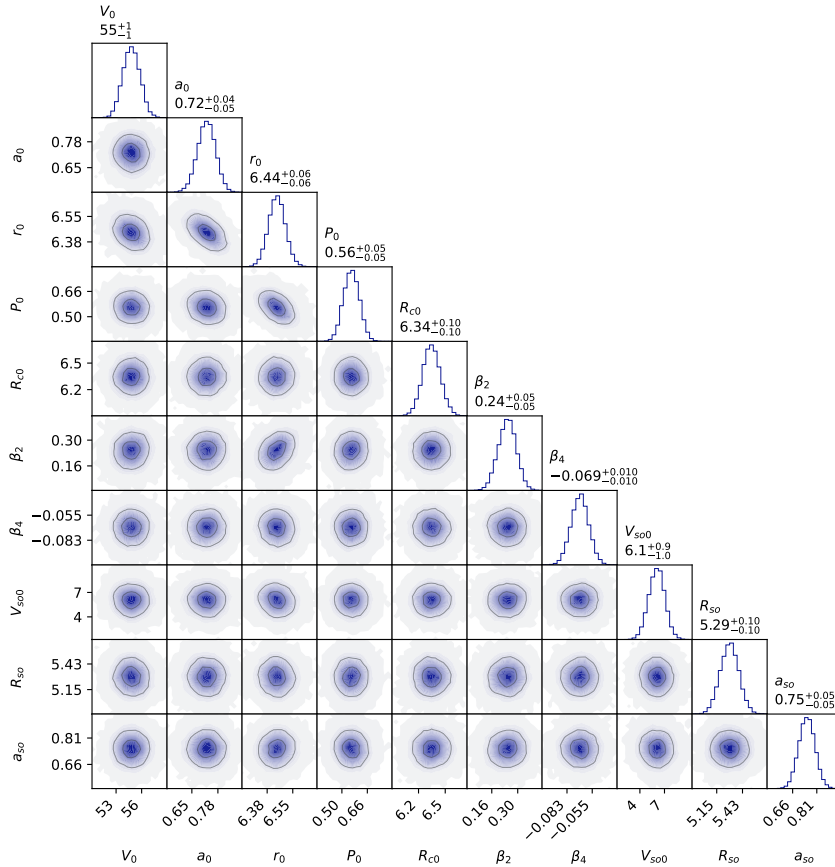


Figure 1.1: Posterior distribution of the parameters for ^{145}Tm proton decay.

上图在 helium1 上计算, 耗时约 3.5d, 速度较慢

后续预计先调整图片样式, 然后做好代码优化加速, 最后可以选取具体核区正式计算
我们使用如下代码修改样式:

```
import prettyplease
from scipy.stats import norm
samples = sampler.get_chain(flat=True)

#a = v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso

labels = ["$V_0$", "$a_0$", "$r_0$", "$P_0$", "$R_{c0}$",
          "$\\beta_2$", "$\\beta_4$", "$V_{so0}$", "$R_{so}$", "$a_{so}$"]
fig, axes = prettyplease.corner(
    samples,
    labels=labels,
```

```

    bins=20,
    colors=['white', '#333333'],
    linewidth=1.0,
    fontsize=10,
    n_ticks=3,
    return_axes=True
)

# ----- 添加 1 区间竖线 -----
ndim = samples.shape[1]      # 参数个数

# 先验参数定义 (与 log_prior 保持一致)
prior_means = np.array([55, 0.75, 6.556, 0.580, 6.342, 0.231, -0.068, 6.2, 5.294, 0.75])
prior_sigmas = np.array([1, 0.05, 0.1, 0.05, 0.1, 0.05, 0.01, 1, 0.1, 0.05])

for i in range(ndim):
    ax = axes[i, i]          # 对角线子图
    data = samples[:, i]

    # 计算 16% 和 84% 分位数 (对应 1 区间边界)
    q16, q84 = np.percentile(data, [16, 84])

    # 绘制两条虚线竖线
    ax.axvline(q16, color='red', linestyle='--', linewidth=1.0, alpha=0.8)
    ax.axvline(q84, color='red', linestyle='--', linewidth=1.0, alpha=0.8)
    median = np.median(data)
    ax.axvline(median, color='blue', linestyle='--', linewidth=1.0, alpha=0.8)

# ----- 叠加先验曲线 -----
for i in range(ndim):
    ax = axes[i, i]
    xlim = ax.get_xlim()
    x_plot = np.linspace(xlim[0], xlim[1], 200)
    pdf = norm.pdf(x_plot, loc=prior_means[i], scale=prior_sigmas[i])
    ax.plot(x_plot, pdf, color='green', linestyle='--', linewidth=1.2, alpha=0.8)
    ax.set_xlim(xlim) # 恢复原范围

# plt.savefig('145Tm.pdf')
plt.savefig('test.pdf')
plt.show()

```

示例图为

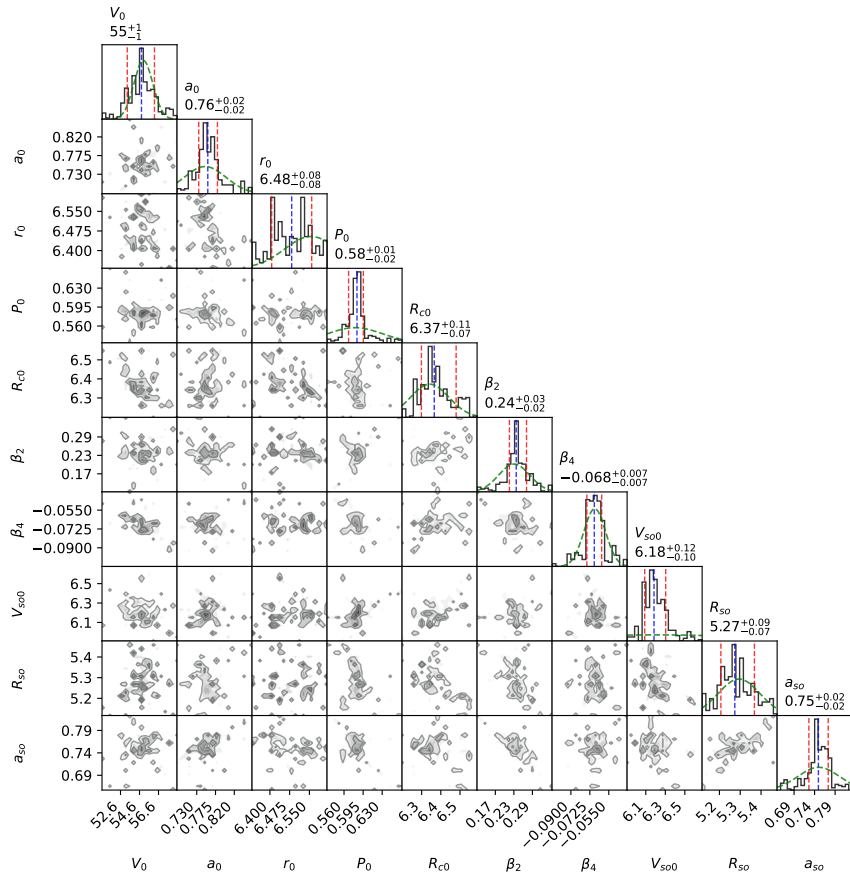


Figure 1.2: 后续计算使用的后验图的示例，包括增加了 prior 线，以及 1σ , 2σ , median 的标定，还使用了灰色线图

对代码进行了部分优化后,显著提高了效率,在 helium1 上使用 jupyter notebook 运行,使用参数如下

```
#System
e2=1.43997 ; hbarc=197.3269718 ; amu=931.49432
zp=69 ; Ap=145 ; mass_excess_p=-27.58 ; mp=Ap*amu+mass_excess_p
z1=68 ; A1=144 ; mass_excess_1=-36.61 ; m1=A1*amu+mass_excess_1      #1-> daughter
z2=1 ; A2=1 ; mass_excess_2=7.288971064 ; m2=A2*amu+mass_excess_2
Q=mp-m1-m2
mu=m1*m2/(m1+m2)
P0=0.580
#Potential

#Deformation
beta2 = 0.231 # quadrupole
beta4 = -0.068 # hexadecapole

#Polarizaton
beta2tilde = 0
beta4tilde = 0

#spin-orbit
Vso0=6.2 ; rso=1.01*(A1)**(1/3) ; aso=0.75 ; lambda_pi=np.sqrt(2)
L=5; S=0.5; J=5.5

#Vn
a0=0.75 ; r0=1.27*A1**(1/3)-0.1 ; V0=55

#Vc
Rc0 = 1.21 * A1**(1/3)

#mesh
r = np.linspace(0.03, 30.0, 1000) # avoid too small r

# cached meshes for repeated gamma evaluations
ROOT_SEARCH_GRID = np.linspace(0.1, 100.0, 200)
R_GRID = np.linspace(0.05, 100.0, 600)
V_CENT_GRID = hbarc**2 * (L + 0.5)**2 / (2 * mu * R_GRID**2)
VC_THETA_GRID = np.linspace(0, np.pi / 2, 100)
VC_SIN_THETA = np.sin(VC_THETA_GRID)
VC_YLO_THETA = {
    lam: sph_harm_y(lam, 0, VC_THETA_GRID, 0).real for lam in (0, 2, 4)
}
VC_YLO_SIN_THETA = {
```

```

    lam: VC_YLO_THETA[lam] * VC_SIN_THETA for lam in (0, 2, 4)
}
Y00 = float(sph_harm_y(0, 0, 0.0, 0).real)
ASSAULT_QUAD_POINTS = 64
ACTION_QUAD_POINTS = 64
ROOT_XTOL = 1e-6
ROOT_ATOL = 1e-5

# experiment data
y=np.array([3.16e-6])    # s
sigma=np.array([0.16e-6])

#a = v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso
def log_prior(a):
    v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso=a
    R=1

# Hyperparameters
v0_mean=55; a0_mean=0.75; r0_mean=6.556; p0_mean=0.580
rc0_mean=6.342; beta2_mean=0.231; beta4_mean=-0.068
vso0_mean=6.2; rso_mean=5.294; aso_mean=0.75

sigma_v0=1*R; sigma_a0=0.05*R; sigma_r0=0.1*R; sigma_p0=0.05*R
sigma_rc0=0.1*R; sigma_beta2=0.05*R; sigma_beta4=0.01*R
sigma_vso0=1*R; sigma_rso=0.1*R; sigma_aso=0.05*R

with multiprocessing.Pool() as pool:
    sampler = emcee.EnsembleSampler(nwalkers, M, log_posterior,
    args=[y,sigma],a=1.0,pool=pool)
    state = sampler.run_mcmc(initial_positions, 200)
    sampler.reset()
    sampler.run_mcmc(state,5000)

```

采样时间 (不算画图和计算 sigma 区间的时间) 约为 25min 获得 145Tm 的后验图如下 (5000/sampler 的试运行)

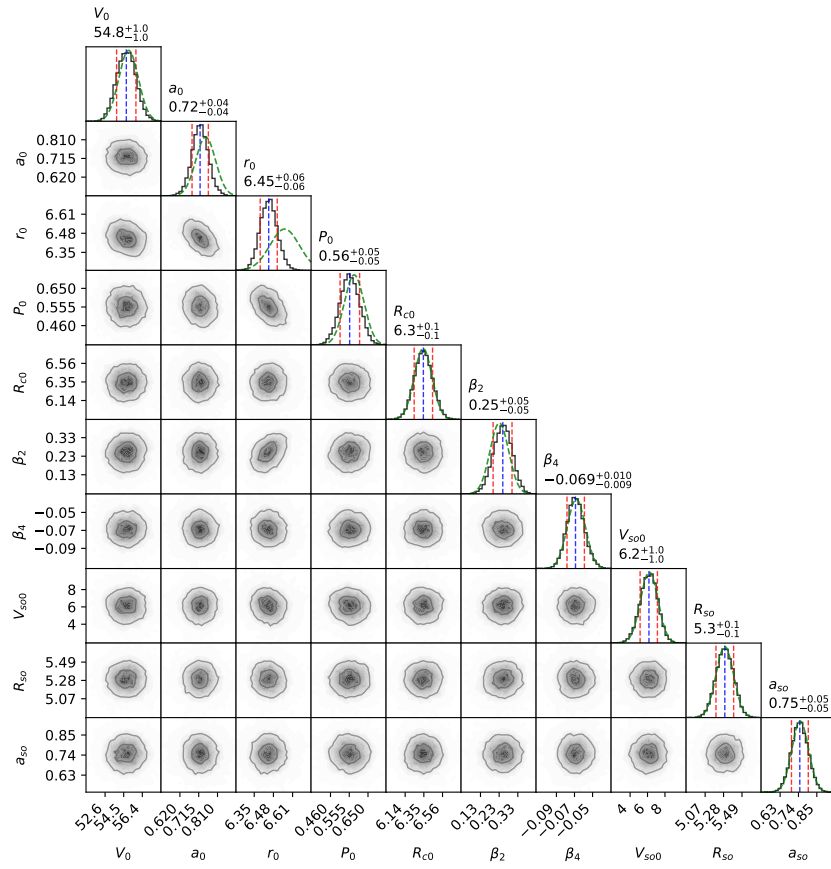


Figure 1.3: 145Tm 的后验图 (5000/sampler 的试运行)

以及对应的 confidence interval:

```
===== Posterior Predictive Distribution of Lifetime =====
Number of posterior samples used: 5000
Mean lifetime      : 3.114e-06 s
Median lifetime    : 3.111e-06 s
Standard deviation  : 1.568e-07 s
1 error bar (±)    : 1.568e-07 s
2 error bar (±)    : 3.137e-07 s
68% CI (1)        : [2.965e-06, 3.268e-06] s
95% CI (2)        : [2.811e-06, 3.429e-06] s
=====
```

实验值在 confidence interval 内, 说明模型的解释性没问题, 代码运行也没问题

计算范式逻辑, 除了可观测量的 sigma 区间, 还包括了敏感性定量描述图, 以及关联矩阵。
示例图如下:

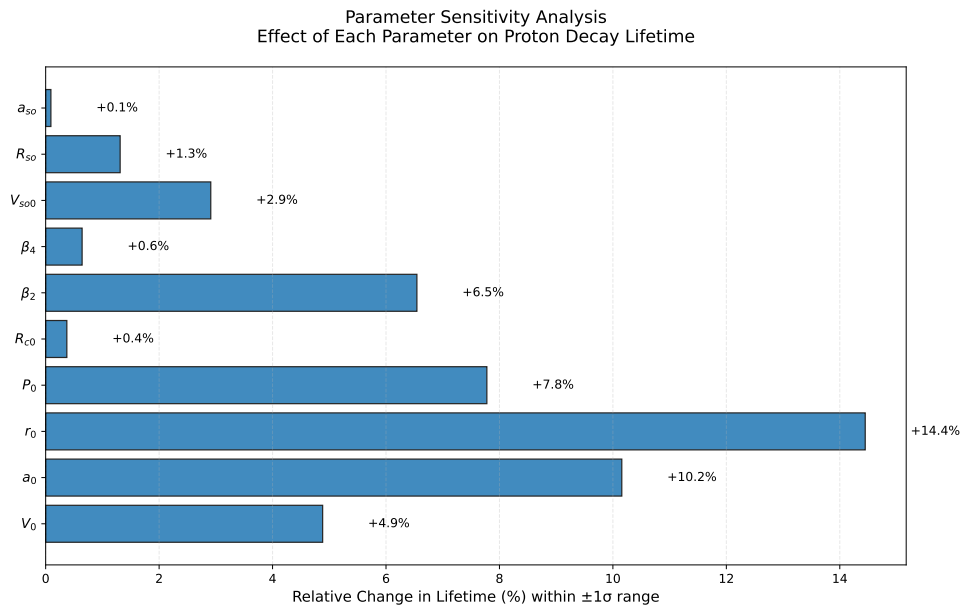


Figure 1.4: 参数敏感性图, 横轴为参数, 纵轴为对应参数的敏感性指标, 红色竖线为实验值, 蓝色竖线为模型预测值, 灰色区域为模型预测的 sigma 区间。

(上图使用了 pandas 和 seaborn 两个 python 包)

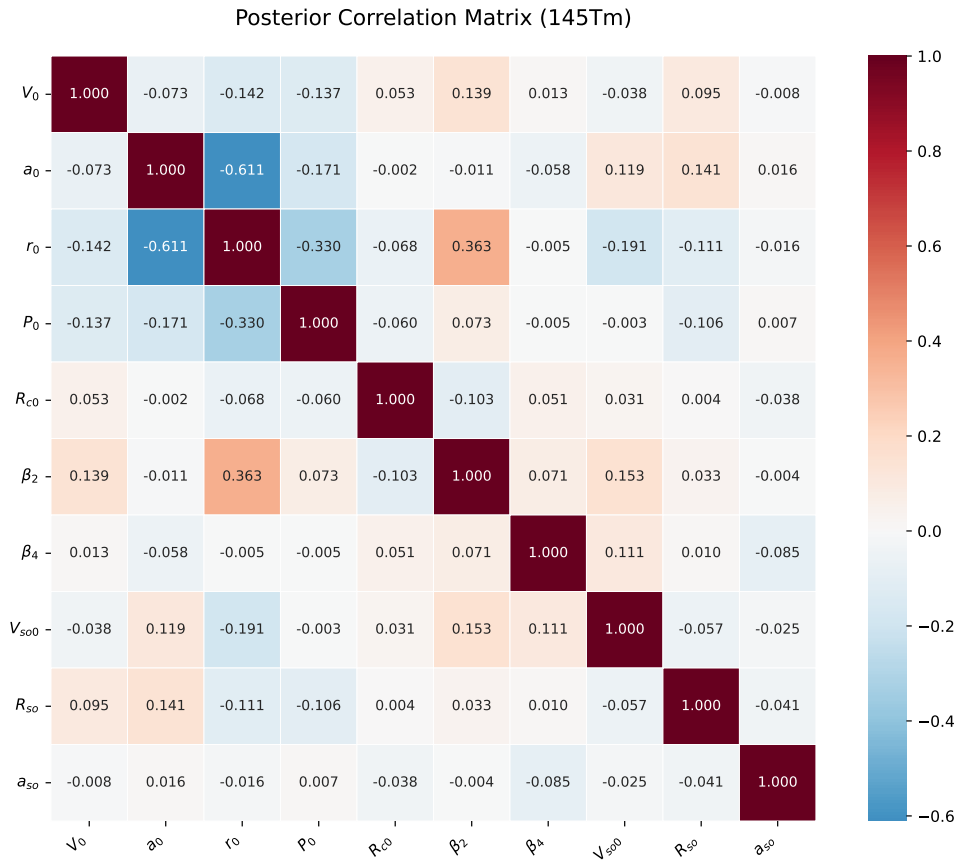


Figure 1.5: 关联矩阵

1.2 正式运行

这里打算研究远离质子幻数的区域，考虑质子幻数在 50-82 之间的质子衰变的核素（大部分选取了 odd-odd 或 odd-A 核）

从 ^{145}Tm 开始, 进行正式运行, 对 ^{145}Tm 选取计算参数

```
#System
e2=1.43997 ; hbarc=197.3269718 ; amu=931.49432
zp=69 ; Ap=145 ; mass_excess_p=-27.58 ; mp=Ap*amu+mass_excess_p
z1=68 ; A1=144 ; mass_excess_1=-36.61 ; m1=A1*amu+mass_excess_1      #1-> daughter
z2=1 ; A2=1 ; mass_excess_2=7.288971064 ; m2=A2*amu+mass_excess_2
Q=mp-m1-m2
mu=m1*m2/(m1+m2)
P0=0.580
#Potential

#Deformation
beta2 = 0.231 # quadrupole
beta4 = -0.068 # hexadecapole

#Polarization
beta2tilde = 0
beta4tilde = 0

#spin-orbit
Vso0=6.2 ; rso=1.01*(A1)**(1/3) ; aso=0.75 ; lambda_pi=np.sqrt(2)
L=5; S=0.5; J=5.5

#Vn
a0=0.75 ; r0=1.27*A1**(1/3)-0.1 ; V0=55

#Vc
Rc0 = 1.21 * A1**(1/3)

#mesh
r = np.linspace(0.03, 30.0, 1000) # avoid too small r

# cached meshes for repeated gamma evaluations
ROOT_SEARCH_GRID = np.linspace(0.1, 100.0, 200)
R_GRID = np.linspace(0.05, 100.0, 600)
V_CENT_GRID = hbarc**2 * (L + 0.5)**2 / (2 * mu * R_GRID**2)
VC_THETA_GRID = np.linspace(0, np.pi / 2, 100)
VC_SIN_THETA = np.sin(VC_THETA_GRID)
VC_YLO_THETA = {
```

```

    lam: sph_harm_y(lam, 0, VC_THETA_GRID, 0).real for lam in (0, 2, 4)
}
VC_YLO_SIN_THETA = {
    lam: VC_YLO_THETA[lam] * VC_SIN_THETA for lam in (0, 2, 4)
}
Y00 = float(sph_harm_y(0, 0, 0.0, 0).real)
ASSAULT_QUAD_POINTS = 64
ACTION_QUAD_POINTS = 64
ROOT_XTOL = 1e-6
ROOT_ATOL = 1e-5

# experiment data
y=np.array([3.16e-6])    # s
sigma=np.array([0.16e-6])

#a = v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso
def log_prior(a):
    v0,a0,r0,p0,rc0,beta2,beta4,vso0,rso,aso=a
    R=1

# Hyperparameters
v0_mean=55; a0_mean=0.75; r0_mean=6.556; p0_mean=0.580
rc0_mean=6.342; beta2_mean=0.231; beta4_mean=-0.068
vso0_mean=6.2; rso_mean=5.294; aso_mean=0.75

sigma_v0=1*R; sigma_a0=0.05*R; sigma_r0=0.1*R; sigma_p0=0.05*R
sigma_rc0=0.1*R; sigma_beta2=0.05*R; sigma_beta4=0.01*R
sigma_vso0=1*R; sigma_rso=0.1*R; sigma_aso=0.05*R

with multiprocessing.Pool() as pool:
    sampler = emcee.EnsembleSampler(nwalkers, M, log_posterior,
    args=[y,sigma],a=2.0,pool=pool)
    state = sampler.run_mcmc(initial_positions, 200)
    sampler.reset()
    sampler.run_mcmc(state,5000)

```

得到的 posterior 图
其参数敏感性分析如下
参数关联图如下

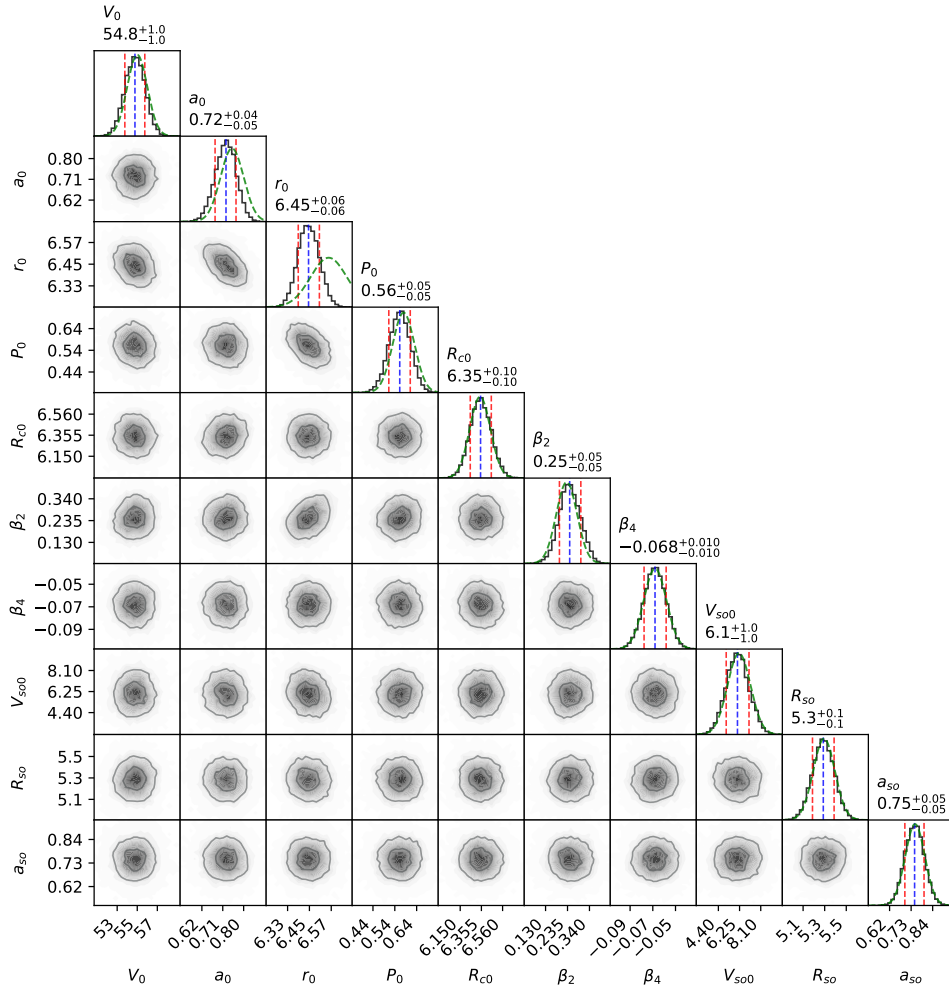


Figure 1.6: 145Tm 的后验图 (5000samples/sampler 的正式运行)

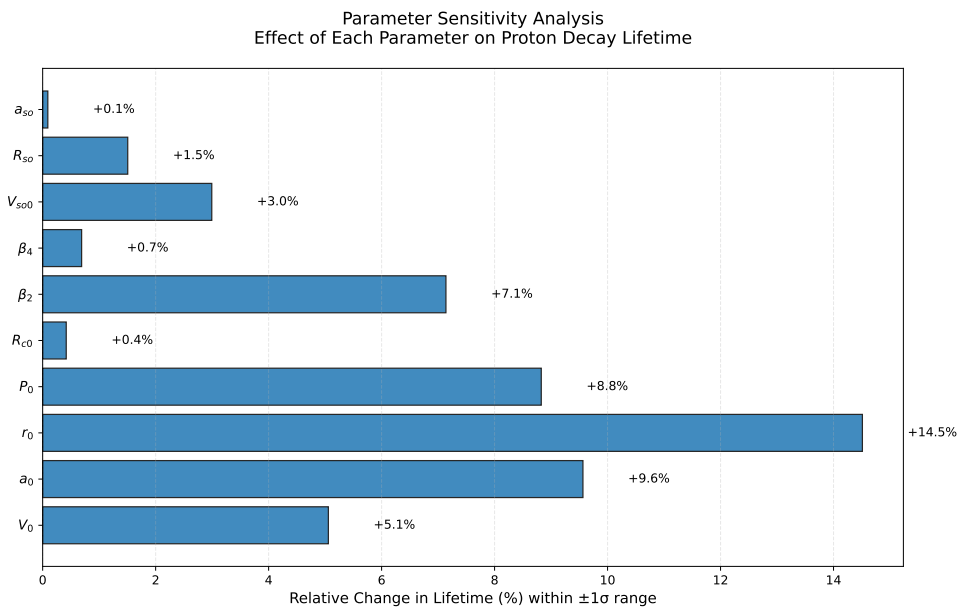


Figure 1.7: 145Tm 的参数敏感性图 (1sigma 区间内的参数变化对半衰期的影响)

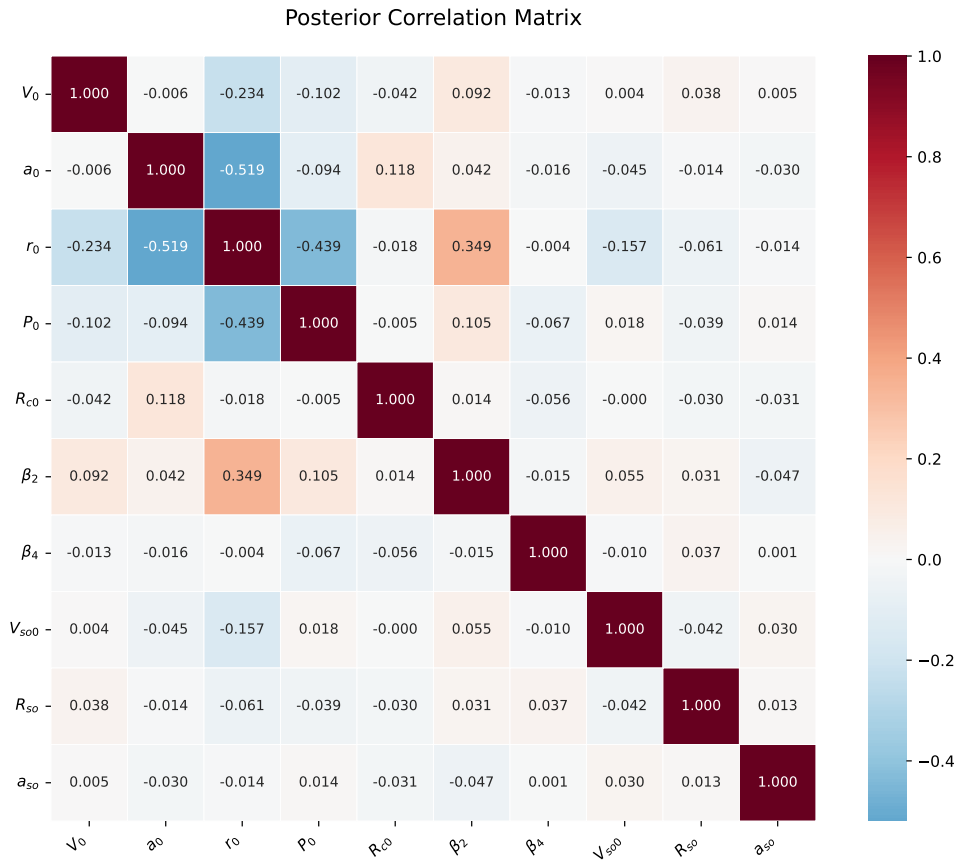


Figure 1.8: ^{145}Tm 的形变势能参数的关联矩阵

^{145}Tm 可观测量的 σ 区间如下 (为计算方便, 从很多 samples 中间均匀抽取 5000 个进行计算)

```
===== Posterior Predictive Distribution of Lifetime =====
Experiment data: 3.16e-6 s
Number of posterior samples used: 5000
Mean lifetime      : 3.109e-06 s
Median lifetime    : 3.109e-06 s
Standard deviation : 1.624e-07 s
1 error bar ( $\pm$ )   : 1.624e-07 s
2 error bar ( $\pm$ )   : 3.248e-07 s
68% CI (1)         : [2.945e-06, 3.274e-06] s
95% CI (2)         : [2.797e-06, 3.424e-06] s
=====
```

然后对 ^{109}I 进行研究

对于 ^{109}I 这个核素, 选取计算参数如下:

```
#System of Proton Decay
e2=1.43997 ; hbarc=197.3269718 ; amu=931.49432
zp=53 ; Ap=109 ; mass_excess_p=-57.673 ; mp=Ap*amu+mass_excess_p      #Parent
z1=zp-1 ; A1=Ap-1 ; mass_excess_1=-65.782 ; m1=A1*amu+mass_excess_1  #1-> daughter
z2=1 ; A2=1 ; mass_excess_2=7.288971064 ; m2=A2*amu+mass_excess_2
Q=mp-m1-m2
mu=m1*m2/(m1+m2)
P0=0.726
#Potential

#Deformation
beta2 = 0.139 # quadrupole
beta4 = 0.056 # hexadecapole

#Polarization
beta2tilde = 0
beta4tilde = 0

#spin-orbit
Vso0=6.2 ; rso=1.01*(A1)**(1/3) ; aso=0.75 ; lambda_pi=np.sqrt(2)
L=2; S=0.5; J=1.5
print("rso=",rso)
#Vn
a0=0.75 ; r0=1.27*A1**(1/3)-0.1 ; V0=55
print("r0=",r0)
#Vc
Rc0 = 1.21 * A1**(1/3)
```

```

print("Rc0=",Rc0)
#mesh
r = np.linspace(0.03, 30.0, 1000) # avoid too small r

# experiment data
y=np.array([92.7e-6])    # s
sigma=np.array([0.7e-6])

# Hyperparameters
v0_mean=55; a0_mean=0.75; r0_mean=5.948; p0_mean=0.726
rc0_mean=5.762; beta2_mean=0.139; beta4_mean=0.056
vso0_mean=6.2; rso_mean=4.809; aso_mean=0.75

with multiprocessing.Pool() as pool:
    sampler = emcee.EnsembleSampler(nwalkers, M, log_posterior,
    args=[y,sigma],a=2.0,pool=pool)
    state = sampler.run_mcmc(initial_positions, 200)
    sampler.reset()
    sampler.run_mcmc(state,10000)

获得了 109I 的 posterior:
然后是 109I 的参数敏感性图:
然后是 109I 的参数关联图:
最后是 109I 的可观测量 sigma 区间:

===== Posterior Predictive Distribution of Lifetime =====
experiment data: [9.27e-05] s
Number of posterior samples used: 5000
Mean lifetime      : 9.261e-05 s
Median lifetime    : 9.260e-05 s
Standard deviation : 7.155e-07 s
1 error bar (±)    : 7.155e-07 s
2 error bar (±)    : 1.431e-06 s
68% CI (1)         : [9.190e-05, 9.332e-05] s
95% CI (2)         : [9.120e-05, 9.403e-05] s
=====

```

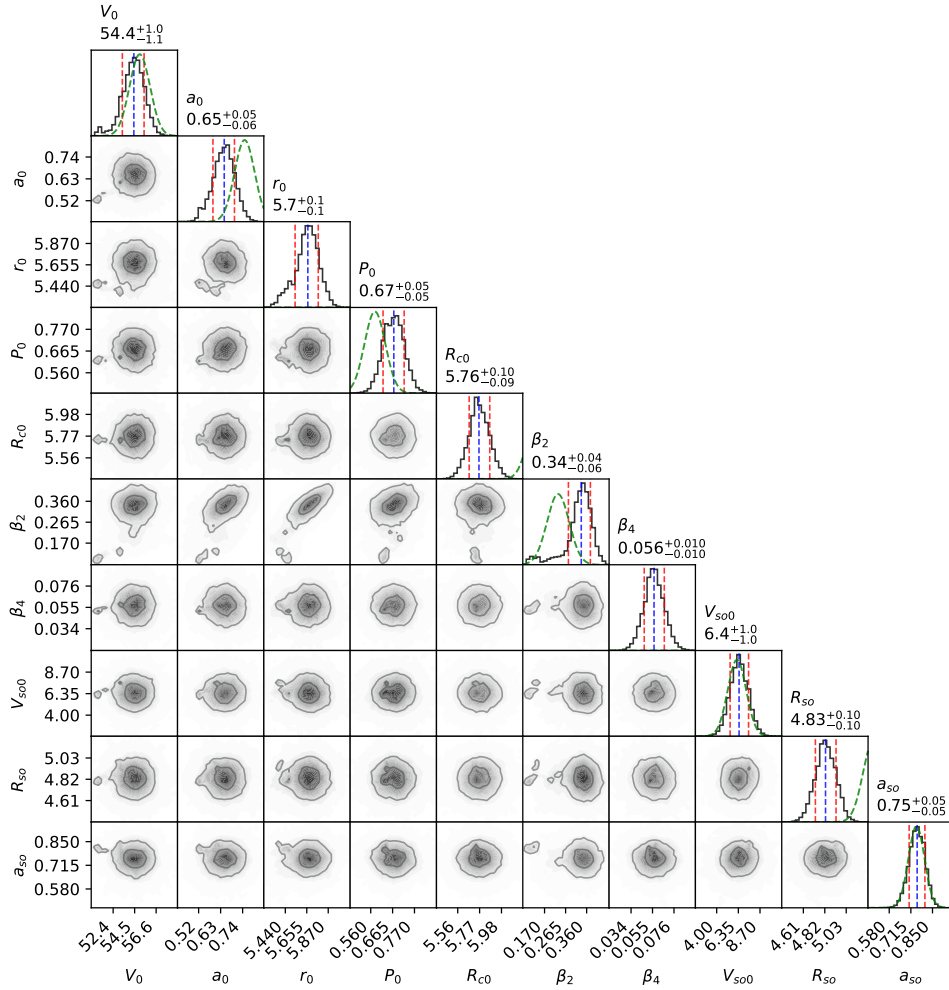


Figure 1.9: 109I 的后验图 (10000samples/sampler 的正式运行)

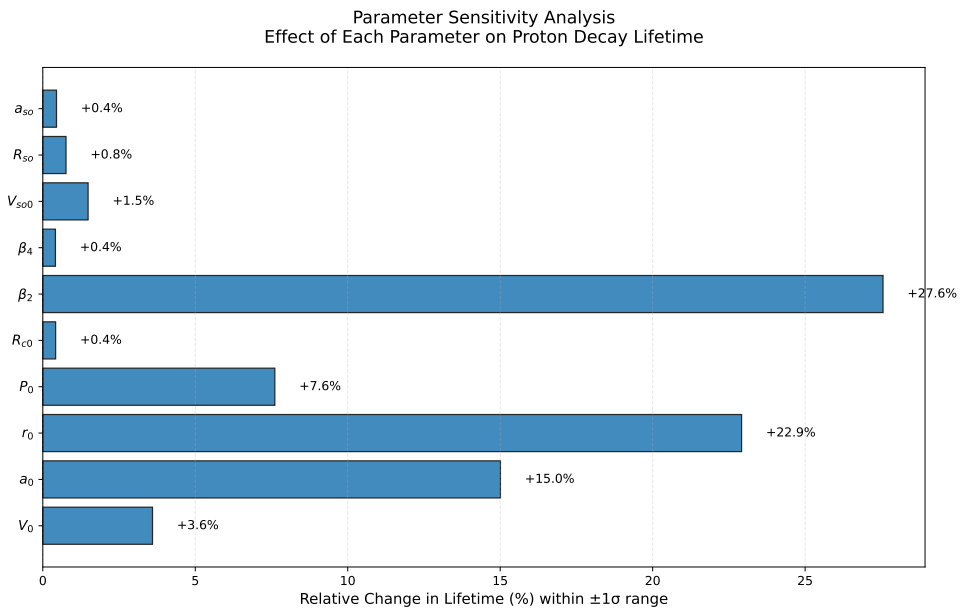


Figure 1.10: 109I 的参数敏感性图 (1sigma 区间内的参数变化对半衰期的影响)

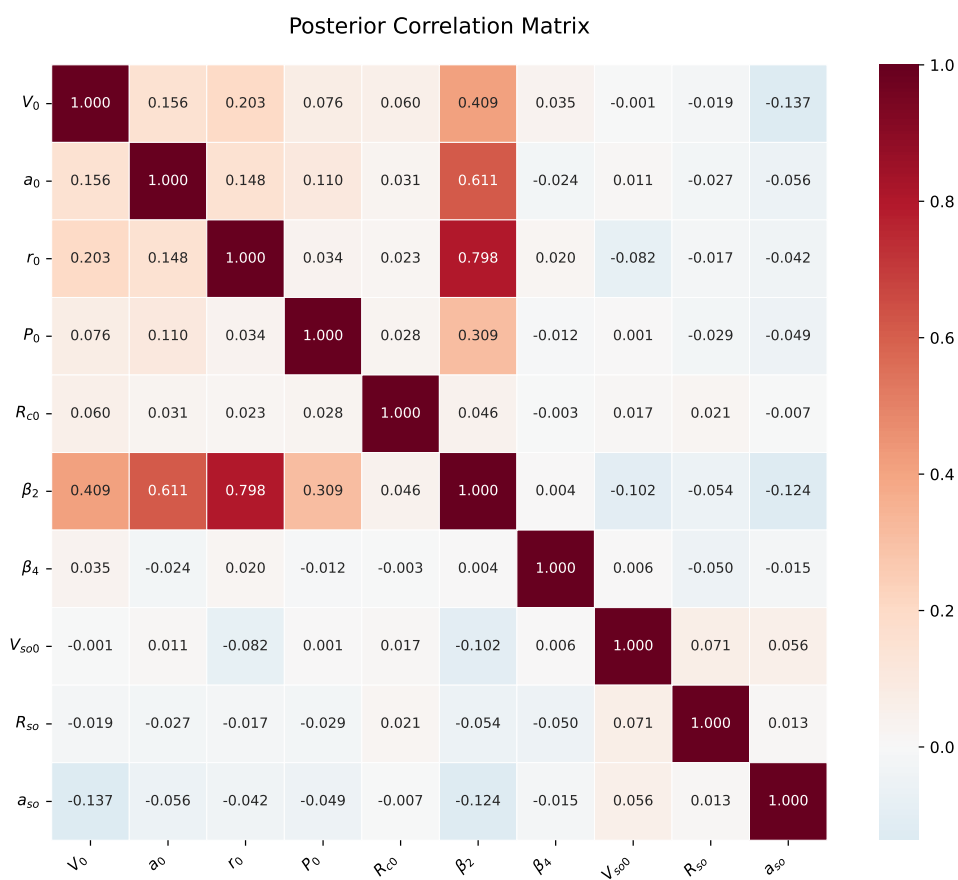


Figure 1.11: 109I 的形变势能参数的关联矩阵

然后是对 ^{113}Cs 的研究计算，对于 ^{113}Cs 选取了计算参数：

```
#System of Proton Decay
e2=1.43997 ; hbarc=197.3269718 ; amu=931.49432
zp=55 ; Ap=113 ; mass_excess_p=-51.765 ; mp=Ap*amu+mass_excess_p      #Parent
z1=zp-1 ; A1=Ap-1 ; mass_excess_1=-60.026 ; m1=A1*amu+mass_excess_1  #1-> daughter
z2=1 ; A2=1 ; mass_excess_2=7.288971064 ; m2=A2*amu+mass_excess_2
Q=mp-m1-m2
print("Q=",Q)
mu=m1*m2/(m1+m2)
P0=0.373
#Potential

#Deformation
beta2 = 0.195 # quadrupole
beta4 = 0.054 # hexadecapole

#Polarization
beta2tilde = 0
beta4tilde = 0

#spin-orbit
Vso0=6.2 ; rso=1.01*(A1)**(1/3) ; aso=0.75 ; lambda_pi=np.sqrt(2)
L=2; S=0.5; J=1.5
print("rso=",rso)
#Vn
a0=0.75 ; r0=1.27*A1**(1/3)-0.1 ; V0=55
print("r0=",r0)
#Vc
Rc0 = 1.21 * A1**(1/3)
print("Rc0=",Rc0)
#mesh
r = np.linspace(0.03, 30.0, 1000) # avoid too small r

# experiment data
y=np.array([16.9e-6]) # s
sigma=np.array([0.1e-6])

# Hyperparameters
v0_mean=55; a0_mean=0.75; r0_mean=6.021; p0_mean=0.373
rc0_mean=5.832; beta2_mean=0.195; beta4_mean=0.054
vso0_mean=6.2; rso_mean=4.868; aso_mean=0.75
```

```

sampler = emcee.EnsembleSampler(nwalkers, M, log_posterior, args=[y,sigma],
a=2.0,pool=pool)
state = sampler.run_mcmc(initial_positions, 500)
sampler.reset()
sampler.run_mcmc(state,10000)

```

这里选择了默认值的采样参数 $a = 2.0$ 以及更多的 burn-in steps 500。

获得的 posterior

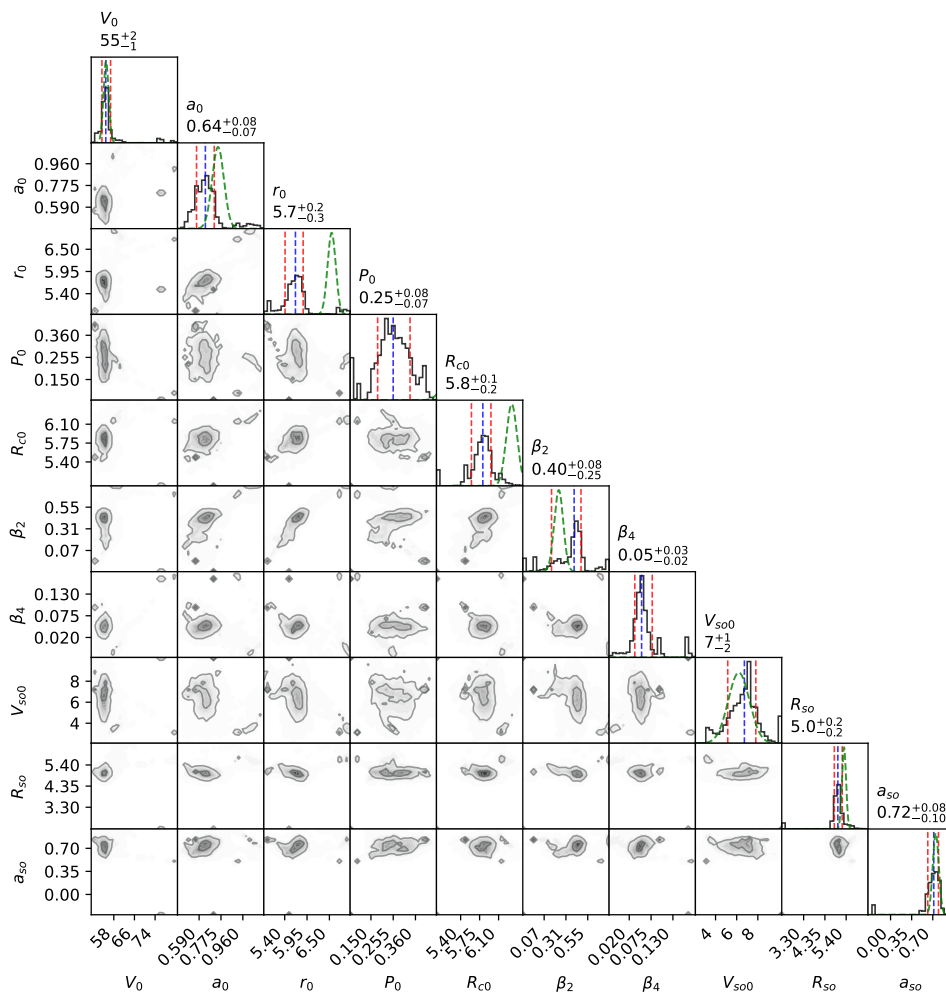


Figure 1.12: 113Cs 的后验图 (10000samples/sampler 的正式运行, 但 burnin step 为 500), 这里获得了一定 correlation

然后是 113Cs 的参数敏感性图:

然后是 113Cs 的参数关联图:

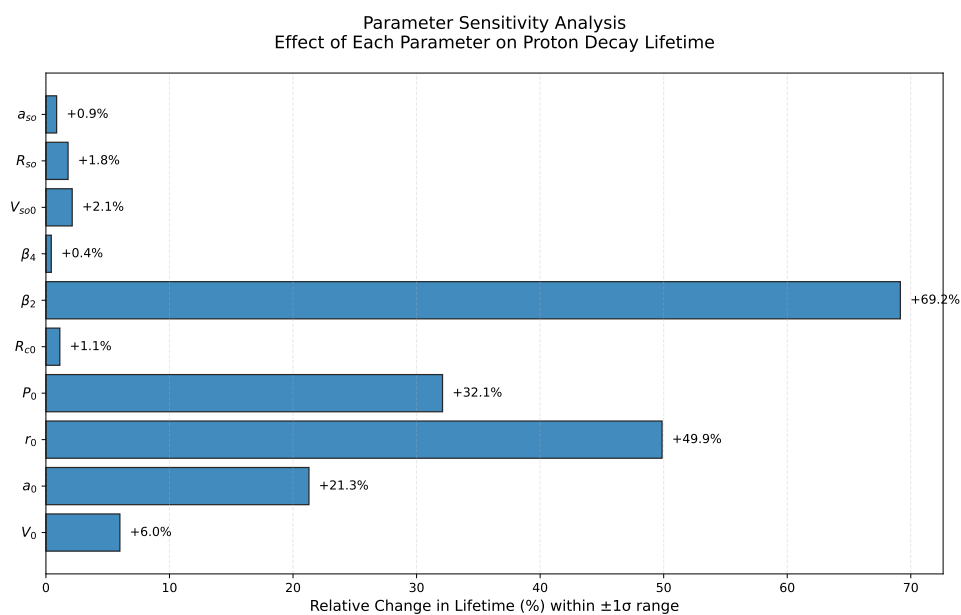


Figure 1.13: 113Cs 的参数敏感性图 (1sigma 区间内的参数变化对半衰期的影响)

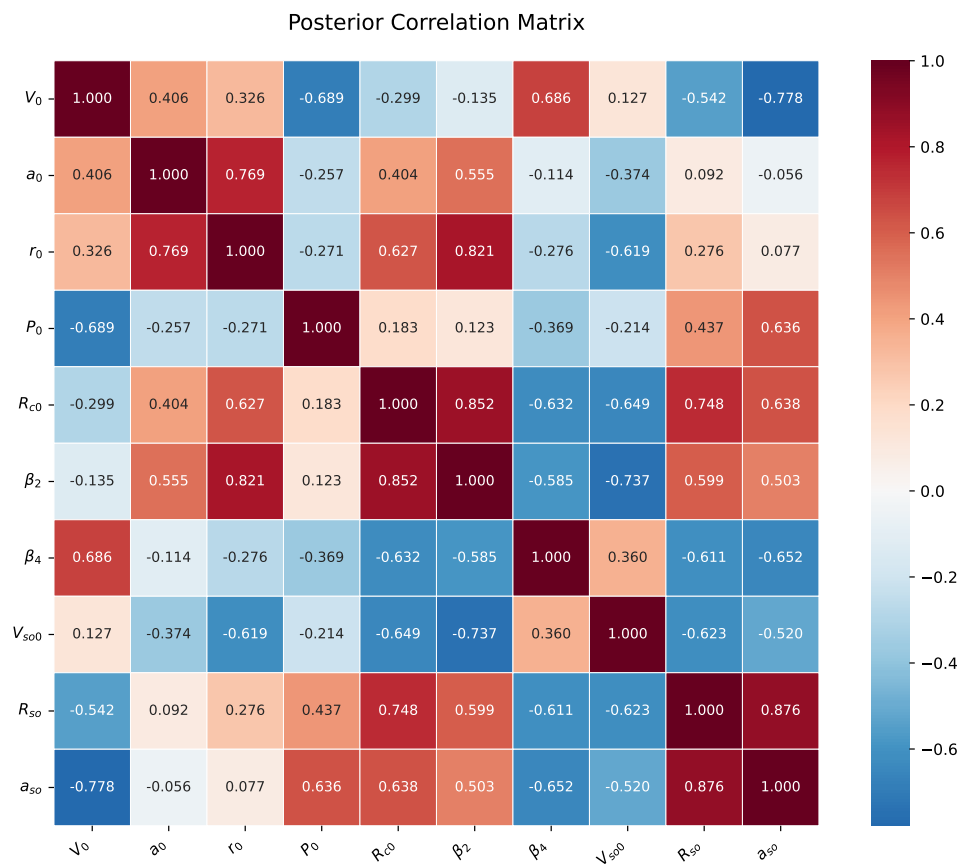


Figure 1.14: 113Cs 的形变势能参数的关联矩阵, 非对角项明显有一定值, 体现了 correlation

最后是 sigma 区间:

```
===== Posterior Predictive Distribution of Lifetime =====  
experiment data: [1.69e-05] s  
Number of posterior samples used: 5000  
Mean lifetime      : 1.685e-05 s  
Median lifetime    : 1.687e-05 s  
Standard deviation : 1.384e-07 s  
1 error bar (±)    : 1.384e-07 s  
2 error bar (±)    : 2.767e-07 s  
68% CI (1)         : [1.673e-05, 1.698e-05] s  
95% CI (2)         : [1.651e-05, 1.708e-05] s  
=====
```

Appendix A

Supplementary Calculations

Put detailed derivations that are too long for the main text here.